COMPUTER SYSTEMS DESIGN PERFORMANCE ANALYSIS

Dr. Gagan Raj Gupta



Ch 13, 14, 15, 22, 23, 24, 26

PERFORMANCE MONITORING TOOLS

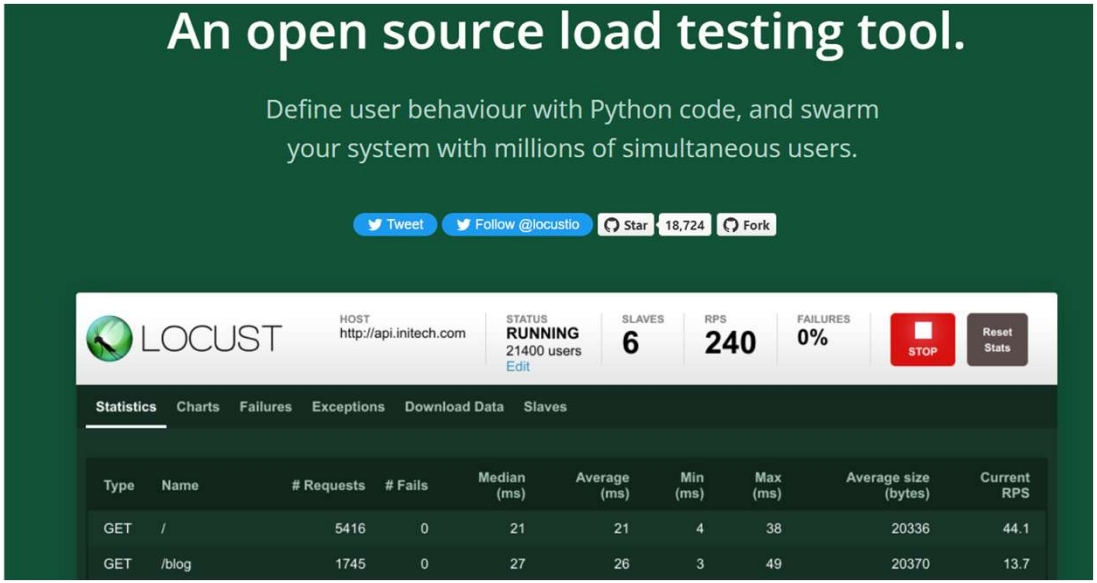
- **top, htop, iotop**
- **vmstat**
- **iostat**
- **lsof [**list of all the open files and the processes. The open files included are **disk files**, **network sockets**, **pipes**, **devices,** and **processes**.**]**
- **tcpdump, netstat,**
- **Nmon** (stands for Nigel's performance Monitor) tool, which is used to monitor all Linux resources such as CPU, Memory, Disk Usage, Network, Top processes, NFS, Kernel, and much more. This tool comes in two modes: Online Mode and Capture Mode.
- **Nagios (Network/Server Monitoring)**: Nagios is a leading open source powerful monitoring system that enables network/system administrators to identify and resolve server-related problems before they affect major business processes.

https://www.tecmint.com/linux-performance-monitoring-tools/



LOAD TESTING TOOLS

- Apache Jmeter:
- K6:
- LOCUST: Python based https://locust.io/



MOTIVATION

- Suppose we have an application/service and want to improve performance
  - This could also be a restaurant like Tech Café …
- How many servers do we need?
  - At what utilization should we keep the servers
  - Is one faster server better than many small servers
- We have many services that interact with each other
  - How much time does my job spend on each of them?
  - Which service is the bottleneck?
- What should be size of the buffers?
- How should the time-out periods be set?
  - Small time-out => more retries (increases load)
  - Large time-out => larger latency



CAPACITY PROVISIONING

- Unfortunately, servers are not free to buy, and not free to operate.
  - Even leaving a server on, but idle, still consumes nearly 60% of the power consumed by a busy server [15].
- Given these costs, it pays to spend some time thinking about how many servers one really needs.
- This subject is called *capacity provisioning*.
- Imagine a customer service center which never has to say "All our reps are busy….."!!
- The purpose is to formulate intuitions and rules of thumb for understanding how many servers we need to achieve a certain QoS goal, and what the impact is of increasing the number of servers.



SCHEDULING

- Suppose you are at a supermarket.
- There are many queues with different number of people ahead of you.
- Which queue will you join?
- We look at each line and count not the number of people (jobs) there, but rather we look at the number of items in each person's basket (the job size) for every person in each line. We then join the line with the smallest total number of items (least work remaining).



MARKOV CHAIN

- A Markov chain or Markov process is **a stochastic model describing a sequence of possible events in which the probability of each event depends only on the state attained in the previous event**.
- **A Markov chain in which every state can be reached from every other state** is called an irreducible Markov chain.
- Limiting Distribution: $\pi_j = \lim_{n\to\infty} (\mathbf{P}^n)_{ij}$, $\vec{\pi} = (\pi_0, \pi_1, \pi_2, \pi_3, \ldots)$ with $\sum_{i=0}^{\infty} \pi_i = 1.$
- Stationary distribution: $\vec{\pi} \cdot \mathbf{P} = \vec{\pi}$ and $\sum_{i=0}^{\infty} \pi_i = 1$
- Assuming the limiting distribution exists, then the limiting distribution is a stationary distribution and no other stationary distribution exists. These theorems are key because they allow us to simply solve the stationary equations to get the limiting distribution.



PERIODICITY OF STATES

Definition 9.1 The ***period*** of state j is the greatest common divisor (GCD) of the set of integers n, such that $P_{jj}^n > 0$. A state is ***aperiodic*** if it has period 1. A chain is said to be aperiodic if all of its states are aperiodic.

Theorem 9.4 *Given an aperiodic, irreducible, finite-state DTMC with transition matrix* $\mathbf{P}$, *as* $n \to \infty$, $\mathbf{P}^n \to \mathbf{L}$, *where* $\mathbf{L}$ *is a limiting matrix all of whose rows are the same vector,* $\vec{\pi}$. *The vector* $\vec{\pi}$ *has all positive components, summing to* 1.



RECURRENCE

Theorem 9.11 *If state i is recurrent, then $\sum_{n=0}^{\infty} P_{ii}^n = \infty$.*
If state i is transient, then $\sum_{n=0}^{\infty} P_{ii}^n < \infty$.

Theorem 9.25 (Ergodic Theorem of Markov Chains) *Given a recurrent, aperiodic, irreducible DTMC, $\pi_j = \lim_{n\to\infty} P_{ij}^n$ exists and*

$$\pi_j = \frac{1}{m_{jj}}, \quad \forall j.$$

For a positive recurrent, aperiodic, irreducible DTMC, $\pi_j > 0$, $\forall_j$.



CONCLUSION

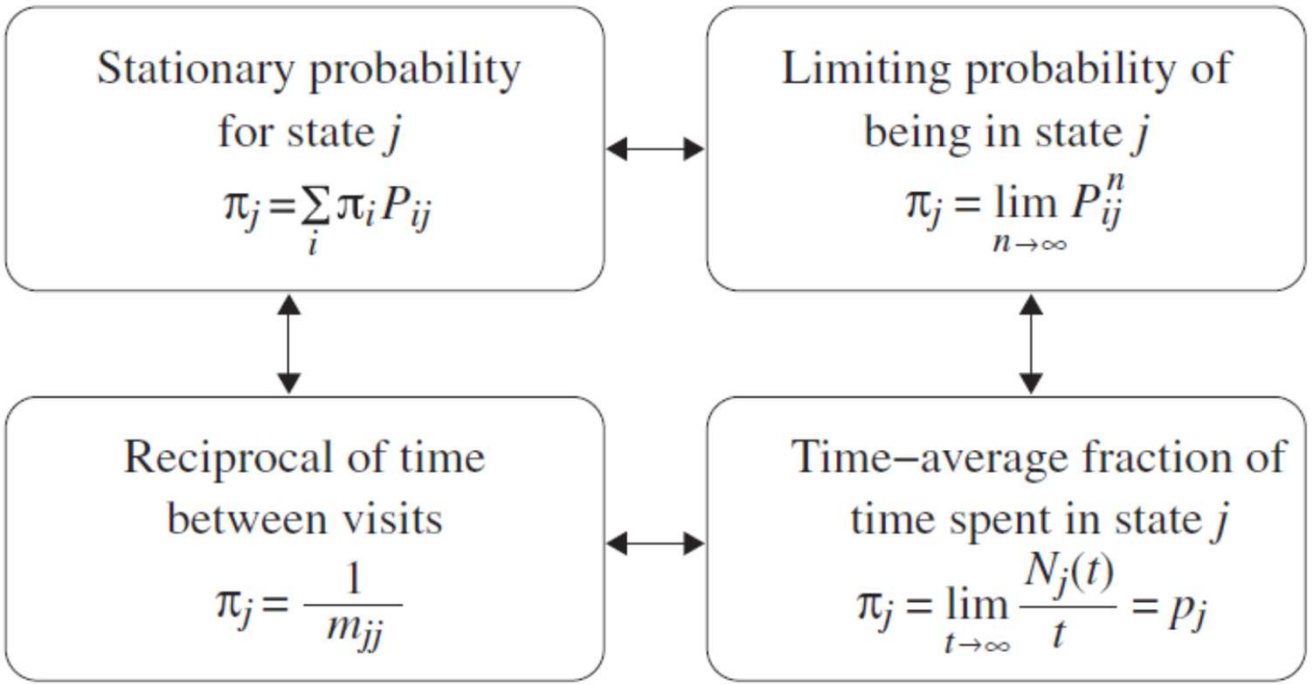


Figure 9.7. Equivalent representations of limiting probabilities.

SUMMARY THEOREM

Theorem 12.6 (Summary Theorem for CTMCs) *Given an irreducible CTMC, suppose* $\exists\ \pi_i$*'s s.t.* $\forall j$,

$$\pi_j \nu_j = \sum_i \pi_i q_{ij} \quad \textit{and} \quad \sum_i \pi_i = 1.$$

Then the π_i*'s are the limiting probabilities for the CTMC, and the CTMC is ergodic.*



M/M/1 SYSTEM

Arrival process is poisson with rate λ

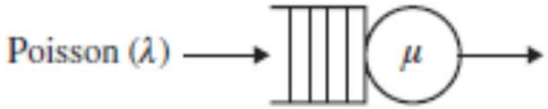


Figure 13.1. The M/M/1 queueing system.

Service time i.i.d.
Exp. R.v. with mean $\frac{1}{\mu}$

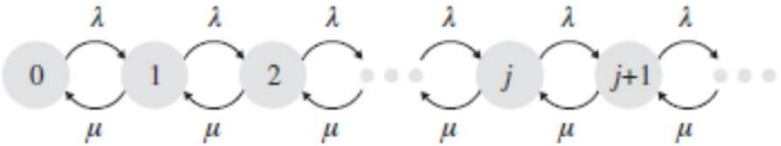


Figure 13.2. CTMC for the M/M/1 queueing system.

State is the number of customers in the system

- M/?/? -> Poisson arrival process (exponential distribution of inter-arrival times)
- ?/M/? -> Service distribution is exponential (time it takes to serve a job)
- ?/?/1 -> 1 server



SOLUTION OF CTMC FOR M/M/1

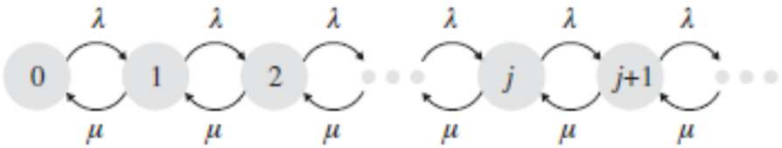


Figure 13.2. CTMC for the M/M/1 queueing system.

State	Balance equation	Simplified equation
$0:$	$\pi_0\lambda = \pi_1\mu$	$\Rightarrow \pi_1 = \frac{\lambda}{\mu}\pi_0$
$1:$	$\pi_1(\lambda+\mu) = \pi_0\lambda + \pi_2\mu$	$\Rightarrow \pi_2 = \left(\frac{\lambda}{\mu}\right)^2 \pi_0$
$2:$	$\pi_2(\lambda+\mu) = \pi_1\lambda + \pi_3\mu$	$\Rightarrow \pi_3 = \left(\frac{\lambda}{\mu}\right)^3 \pi_0$
$i:$	$\pi_i(\lambda+\mu) = \pi_{i-1}\lambda + \pi_{i+1}\mu$	$\Rightarrow ???$

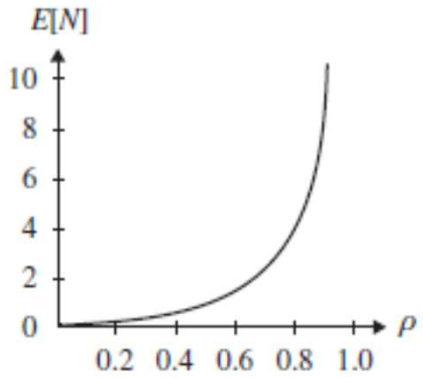
$$\pi_i = \left(\frac{\lambda}{\mu}\right)^i \pi_0.$$

$$\pi_i = \left(\frac{\lambda}{\mu}\right)^i \left(1 - \frac{\lambda}{\mu}\right) = \rho^i(1-\rho)$$

$$\pi_0 = 1 - \frac{\lambda}{\mu} = 1 - \rho$$



QUEUEING



$$\mathbf{Var}(N) = \frac{\rho}{(1-\rho)^2}.$$

$$\begin{aligned}
\mathbf{E}[N] &= \sum_{i=0}^{\infty} i\pi_i \\
&= \sum_{i=1}^{\infty} i\pi_i \\
&= \sum_{i=1}^{\infty} i\rho^i(1-\rho) \\
&= \rho(1-\rho)\sum_{i=1}^{\infty} i\rho^{i-1} \\
&= \rho(1-\rho)\frac{d}{d\rho}\left[\sum_{i=0}^{\infty}\rho^i\right] \\
&= \rho(1-\rho)\frac{d}{d\rho}\left[\frac{1}{1-\rho}\right] \\
&= \rho(1-\rho)\frac{1}{(1-\rho)^2} \\
&= \frac{\rho}{1-\rho}.
\end{aligned}$$

$$\mathbf{E}[T] = \frac{\mathbf{E}[N]}{\lambda} = \frac{1}{\mu-\lambda}$$

$$\mathbf{E}[T_Q] = \mathbf{E}[T] - \frac{1}{\mu} = \frac{\rho}{\mu-\lambda}$$

$$\mathbf{E}[N_Q] = \frac{\rho^2}{1-\rho}.$$

DistributionofResponseTime
$T \sim \mathrm{Exp}(\mu - \lambda$



EXAMPLE

- At Tech Café the service times are exponentially distributed. The chef can only work on preparing a single order at a time. An average customer at the Tech Café takes 5 minutes to be served. The arrival process is Poisson as there are large number of students who visit it at various times of the day. We want to keep the mean waiting time under 10 minutes. What is the maximum arrival rate that can be sustained?
- **Solution:** $\frac{1}{\mu-\lambda} < 10 \Rightarrow 1 < 10(0.2-\lambda) \Rightarrow 10\lambda < 1 \Rightarrow \lambda < 0.1$
  - Only 1 person per 10 minutes on average
  - Suppose λ=0.05; the average waiting time would be $\frac{1}{0.2-0.05} = \frac{1}{0.15} = 6.66\ minutes$
  - We need to increase the service rate!!



LET'S INCREASE THE SERVICE RATE

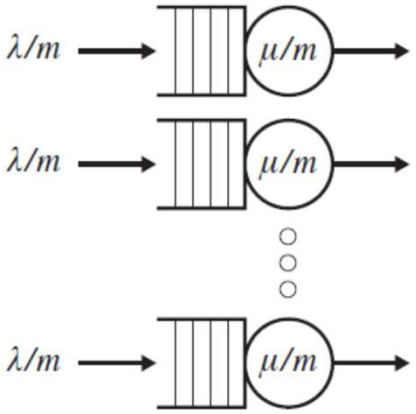
- Let's increase the service rate from 0.2/min to 1/min
- Let's also increase the arrival rate to 0.25
- Both have increase 5 times.
- What is the new avg. waiting time?
- It is 5 times less…
- **Solution:** $\frac{1}{1-0.25} = \frac{1}{0.75} = 1.33\ minutes$



SINGLE QUEUE OR MULTIPLE QUEUES?



Vs.



$$\mathbf{E}\left[T^{\mathrm{SM}}\right] = \frac{1}{\mu - \lambda}.$$

$$\mathbf{E}\left[T^{\mathrm{FDM}}\right] = \frac{1}{\mu/m - \lambda/m} = \frac{m}{\mu - \lambda}.$$

- Frequency-division multiplexing guarantees a specific service rate to each stream. Statistical multiplexing is unable to provide any such guarantee.
- More importantly, suppose the original *m* streams were very regular (i.e., the interarrival times were less variable than Exponential, say closer to Deterministic than Exponential). By merging the streams, we introduce lots of variability into the arrival stream. This leads to problems if an application requires a low variability in delay (e.g., voice or video).



FINITE WAITING SPACE (M/M/1/N)

- If we have a finite seating space (buffer), then some customers will be turned away.
- Probability of having "i" customers waiting reduces as "i" increases

$$\pi_i = \left(\frac{\lambda}{\mu}\right)^i \pi_0.$$

- In Exercise 13.5, you will be able to show that when the buffer is limited to N-1;

$$\pi_i = \frac{\rho^i(1-\rho)}{1-\rho^{N+1}}$$

- As real estate is expensive, one can see that there is no point in having a very large seating space!
- In Exercise 13.5, we raise the question of whether the service rate should be doubled or buffer space should be doubled. The answer depends on the utilization!



TIME REVERSIBILITY

Definition 14.1 A CTMC is ***time-reversible*** if, for all states i and j, the rate of transitions from state i to state j equals the rate of transitions from state j to state i (i.e., $\pi_i q_{ij} = \pi_j q_{ji}$, where $\sum_i \pi_i = 1$).

Lemma 14.2 *Given an irreducible CTMC, suppose we can find x_i's such that*

$$\sum_i x_i = 1 \quad \textit{and} \quad x_i q_{ij} = x_j q_{ji}, \quad \forall i, j$$

where q_{ij} is the rate of transitions from state i to state j given that the MC is in state i. Then,

1. *The x_i's are the limiting probabilities of the CTMC.*
2. *The CTMC is time-reversible.*



Question: Recall that the M/M/1 chain is a birth-death process. Are all birth-death processes time-reversible?

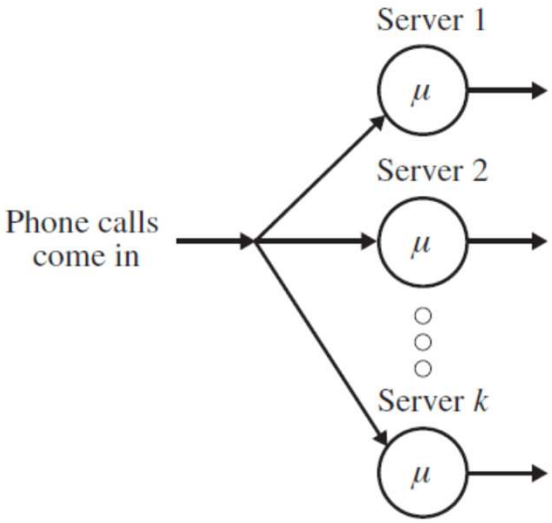
Answer: Yes. Here's a proof: First observe that during any period of time, t, the number of transitions from state i to state $i+1$ is within 1 of the number of transitions from state $i+1$ to state i. The reason for this is that you cannot repeat going from i to $i+1$ without first going back to i again – and the only way to go back to state i is to make a transition from $i+1$ to i. Thus the long-run rate of transitions (number of transitions divided by time) from state i to state $i+1$ is equal to the rate of transitions from $i+1$ to i (as time gets big, that "difference of 1" can be ignored).

As in the case of DTMCs, it is often helpful to write the time-reversibility equations and see if a solution to these can be found. If so, that solution also represents the limiting distribution. If not, then one needs to go back to the balance equations. Fortunately, we will see that the CTMCs for the M/M/k/k and the M/M/k are both birth-death processes, and hence the time-reversibility equations will be solvable.



M/M/K/K

What fraction of jobs are lost in this system?



- K-server loss system
- The M/M/k/k loss system originally arose from the early phone switching systems that could handle at most *k* calls simultaneously.
- An incoming call request could be picked up and serviced by any one of the *k* circuits.
- However, if none of the *k* circuits was free, the phone call request was dropped. The duration of a phone call was assumed to be Exponentially distributed.
- Another application for the *k*-server loss system is a system that maintains virtual connections between nodes A and B in a network. Only *k* virtual connections are allowed. Each incoming request for a virtual connection is given one; however, if all *k* virtual connections are in use, the request is rejected



CTMC

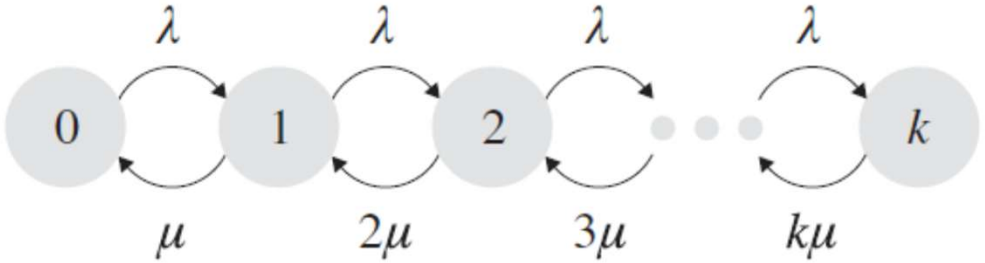
$$\pi_i = \left(\frac{\lambda}{\mu}\right)^i \frac{1}{i!}\pi_0.$$

$$\sum_{i=0}^{k} \pi_i = 1$$

$$\sum_{i=0}^{k} \left(\frac{\lambda}{\mu}\right)^i \frac{1}{i!}\pi_0 = 1$$

$$\Rightarrow \pi_0 = \frac{1}{\sum_{i=0}^{k} \left(\frac{\lambda}{\mu}\right)^i \frac{1}{i!}}$$

$$\pi_i = \frac{\left(\frac{\lambda}{\mu}\right)^i / i!}{\sum_{j=0}^{k} \left(\frac{\lambda}{\mu}\right)^j \frac{1}{j!}}.$$



State	Time-reversibility equation	Simplified equation
0	$\pi_0\lambda = \pi_1\mu$	$\pi_1 = \frac{\lambda}{\mu}\pi_0$
1	$\pi_1\lambda = \pi_2 2\mu$	$\pi_2 = \left(\frac{\lambda}{\mu}\right)^2 \frac{1}{2!}\pi_0$
2	$\pi_2\lambda = \pi_3 3\mu$	$\pi_3 = \left(\frac{\lambda}{\mu}\right)^3 \frac{1}{3!}\pi_0$
$k-1$	$\pi_{k-1}\lambda = \pi_k k\mu$	$\pi_k = \left(\frac{\lambda}{\mu}\right)^k \frac{1}{k!}\pi_0$

$$P_{\text{block}} = \pi_k = \frac{\left(\frac{\lambda}{\mu}\right)^k / k!}{\sum_{j=0}^{k} \left(\frac{\lambda}{\mu}\right)^j \frac{1}{j!}}.$$

Erlang-B formula

INDEPENDENCE FROM SERVICE DIST.

- Erlang-B formula is *independent of the service time distribution.*
  - *ARRIVALS NEED TO BE POISSON!! =➔ M/G/K/K*
  - Same formula arises when the service demand has a mean of $\frac{1}{\mu}$ with any probability distribution
- This is known as an **insensitivity result**, because the result depends only on the mean of the distribution
  - It would have been much more typical that queueing behavior is highly influenced by the distribution of the service time
  - Insensitivity results often occur in situations where *there is no queue*



M/M/K

Table 14.2. *Time-reversibility equations for the M/M/k*

State	Time-reversibility equation	Simplified equation
0	$\pi_0 \lambda = \pi_1 \mu$	$\pi_1 = \frac{\lambda}{\mu}\pi_0$
1	$\pi_1 \lambda = \pi_2 2\mu$	$\pi_2 = \left(\frac{\lambda}{\mu}\right)^2 \frac{1}{2!}\pi_0$
2	$\pi_2 \lambda = \pi_3 3\mu$	$\pi_3 = \left(\frac{\lambda}{\mu}\right)^3 \frac{1}{3!}\pi_0$
$k-1$	$\pi_{k-1} \lambda = \pi_k k\mu$	$\pi_k = \left(\frac{\lambda}{\mu}\right)^k \frac{1}{k!}\pi_0$
k	$\pi_k \lambda = \pi_{k+1} k\mu$	$\pi_{k+1} = \left(\frac{\lambda}{\mu}\right)^{k+1} \frac{1}{k!}\frac{1}{k}\pi_0$
$k+1$	$\pi_{k+1} \lambda = \pi_{k+2} k\mu$	$\pi_{k+2} = \left(\frac{\lambda}{\mu}\right)^{k+2} \frac{1}{k!}\frac{1}{k^2}\pi_0$

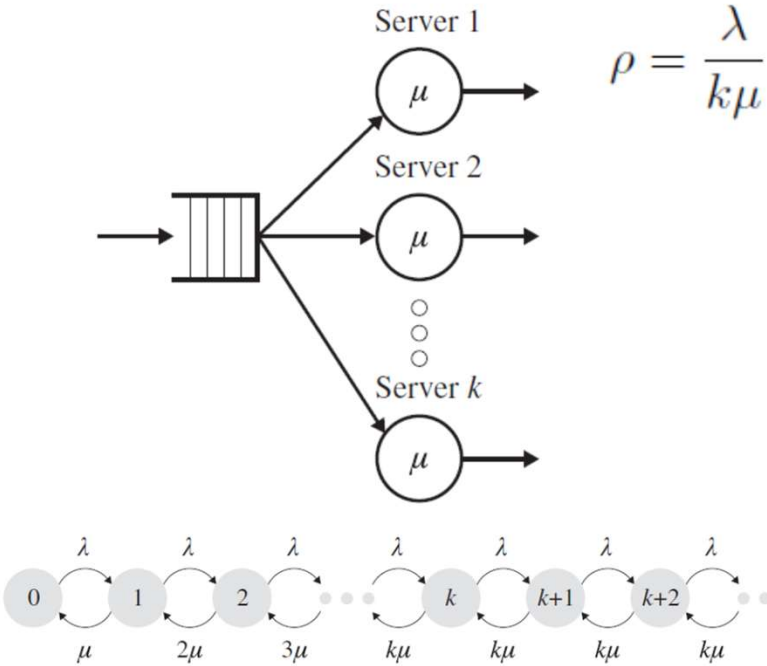


Figure 14.4. An M/M/k queueing system modeled using a CTMC.

$$\pi_i = \begin{cases} \left(\frac{\lambda}{\mu}\right)^i \frac{1}{i!}\pi_0 & \text{if } i \le k \\ \left(\frac{\lambda}{\mu}\right)^i \frac{1}{k!}\left(\frac{1}{k}\right)^{i-k}\pi_0 & \text{if } i > k \end{cases}.$$

M/M/K

$$\rho = \frac{\lambda}{k\mu} \qquad \pi_i = \begin{cases} \frac{(k\rho)^i}{i!}\pi_0 & \text{if } i \leq k \\ \frac{\rho^i}{k!}k^k\pi_0 & \text{if } i > k \end{cases}$$

$$\pi_0 + \sum_{i=1}^{k-1}\pi_i + \sum_{i=k}^{\infty}\pi_i = 1$$

$$\pi_0\left[1 + \sum_{i=1}^{k-1}\frac{(k\rho)^i}{i!} + \sum_{i=k}^{\infty}\frac{\rho^i}{k!}k^k\right] = 1$$

$$\pi_0\left[\sum_{i=0}^{k-1}\frac{(k\rho)^i}{i!} + \frac{k^k}{k!}\frac{\rho^k}{1-\rho}\right] = 1$$

$$\Rightarrow \pi_0 = \left[\sum_{i=0}^{k-1}\frac{(k\rho)^i}{i!} + \frac{(k\rho)^k}{k!(1-\rho)}\right]^{-1}$$



Probability That an Arriving Job Has to Queue

Having found the stationary probabilities, we now find the **probability that an arriving job has to queue, P_Q**. Observe that P_Q is the probability that an arrival finds all k servers busy.

$$
\begin{aligned}
P_Q &= \mathbf{P}\{\text{An arrival finds all servers busy}\} \\
&= \mathbf{P}\{\text{An arrival sees} \geq k \text{ jobs in system}\} \\
&= \text{Limiting probability that there are} \geq k \text{ jobs in system} \quad \text{(by PASTA)} \\
&= \sum_{i=k}^{\infty} \pi_i \\
&= \frac{k^k}{k!}\pi_0 \sum_{i=k}^{\infty} \rho^i \\
&= \frac{(k\rho)^k \pi_0}{k!(1-\rho)} \quad \text{where } \pi_0 = \left[\sum_{i=0}^{k-1} \frac{(k\rho)^i}{i!} + \frac{(k\rho)^k}{k!(1-\rho)}\right]^{-1} \qquad (14.5)
\end{aligned}
$$

Equation (14.5) is the famous Erlang-C formula.



COMPARING M/M/K WITH M/M/K/K

Theorem 14.5 *Let P_{block} denote the blocking probability for the M/M/k/k and P_Q the queueing probability for the M/M/k. Let ρ denote the load (system utilization) for the M/M/k. Then*

$$P_{\text{block}} = \frac{(1-\rho)P_Q}{1-\rho P_Q}. \tag{14.6}$$

$$\mathbf{E}[N_Q] = P_Q \frac{\rho}{1-\rho}.$$



THREE SERVER ORGANIZATIONS

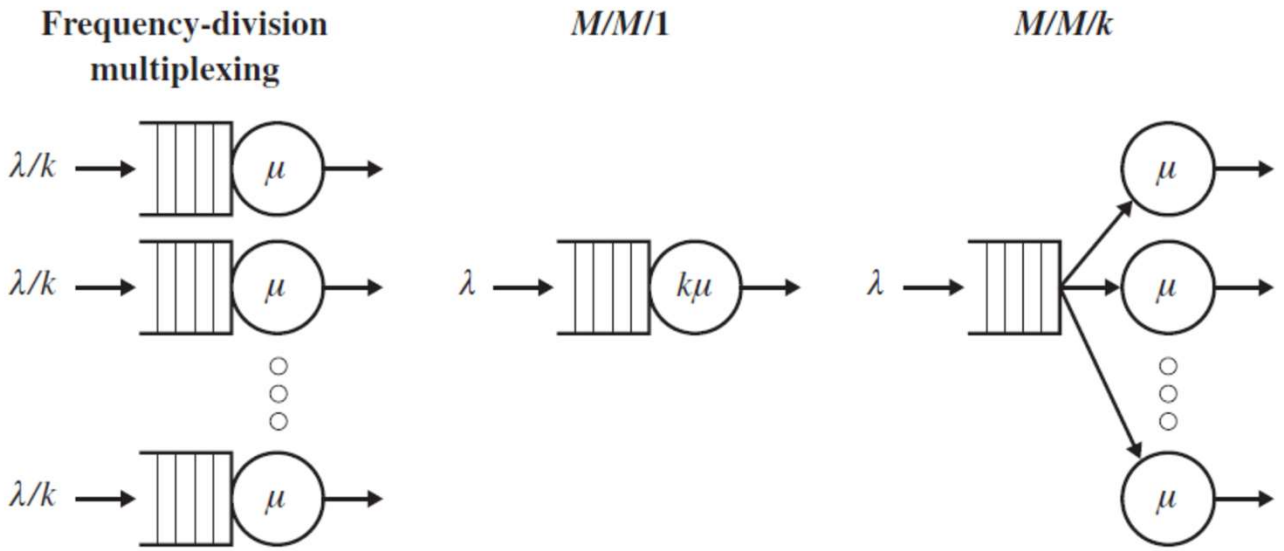


Figure 14.5. Frequency-division multiplexing, M/M/1, and M/M/k, all with load $\rho = \frac{\lambda}{k\mu}$.

$$\mathbf{E}[T]^{\text{FDM}} = \frac{1}{\mu - \frac{\lambda}{k}} = \frac{k}{k\mu - \lambda}. \qquad \mathbf{E}[T]^{\text{M/M/1}} = \frac{1}{k\mu - \lambda}. \qquad \mathbf{E}[T]^{\text{M/M/k}} = \frac{1}{\lambda} \cdot P_Q \cdot \frac{\rho}{1-\rho} + \frac{1}{\mu}$$

To compare the M/M/k with the M/M/1, consider

$$\frac{\mathbf{E}[T]^{\text{M/M/k}}}{\mathbf{E}[T]^{\text{M/M/1}}} = \frac{\frac{1}{\lambda} \cdot P_Q \cdot \frac{\rho}{1-\rho} + \frac{1}{\mu}}{\frac{1}{\lambda} \cdot \frac{\rho}{1-\rho}}$$

$$= P_Q^{\text{M/M/k}} + \frac{\lambda}{\mu} \cdot \frac{1-\rho}{\rho}$$

$$= P_Q^{\text{M/M/k}} + k(1-\rho). \tag{14.13}$$

Now consider two cases.

Case 1: $\rho \approx 0$

As the load drops, the probability of queueing drops, so $P_Q^{\text{M/M/k}} \approx 0$, and expression (14.13) is approximately $0 + k = k$. Thus, the M/M/1 is k times faster than M/M/k.

Question: Explain intuitively why this makes sense.

Answer: In the M/M/k, under light load, most servers are idle. The few servers that are busy serve the jobs they get at rate μ. By contrast, in the M/M/1, with the same light load, every job gets served with rate $k\mu$. Thus jobs complete more quickly in the M/M/1.

Case 2: $\rho \approx 1$

With the load high, $P_Q^{\text{M/M/k}} \approx 1$, and so expression (14.13) is approximately 1. Thus, the M/M/k and M/M/1 have the same response time.



LOAD IN A M/M/K SYSTEM

- A common rule of thumb for a single-server M/M/1 system is that the utilization, ρ, should be kept below 0.8.
  - If ρ gets higher (e.g., $\rho = 0.95$), delays explode.
  - For example, with $\rho = 0.8$, $\mathbf{E}[N]$ for an M/M/1 is 4, whereas for $\rho = 0.95$, $\mathbf{E}[N] = 19$.
- However, in M/M/K high load does not necessarily imply high delay
  - $E[T_Q] = \frac{1}{\lambda} P_Q \frac{\rho}{1-\rho}$ here $\rho = \frac{\lambda}{k\mu}$
  - $\mathrm{E}[T_Q] = E[T_Q|delayed].P_Q = \frac{1}{\lambda} P_Q \frac{\rho}{1-\rho}$
  - $\frac{\mathrm{E}[T_Q]}{P_Q} = E[T_Q|delayed] = \frac{1}{\lambda}\frac{\rho}{1-\rho} = \frac{1}{k\mu(1-\rho)}$
- TAKE HOME MESSAGE: high values of system utilization, ρ, do not imply that customers will suffer, provided that there are sufficiently many servers



LOAD IN A M/M/K SYSTEM

- Assume average server utilization of $\rho = 0.95$ with 5 servers.
  - Average wait for customers who are delayed is $1/5\mu(.05) = 4/\mu$
  - 4 times an average job's size!
- Same average server utilization of $\rho = 0.95$ with 100 servers.
  - Average wait for customers who are delayed is only $1/100\mu(.05) = 1/5\mu$
  - *A* fifth of an average job's size.
- Even if all servers still have utilization ρ, with more servers it is less likely that they are *all* busy at the same time.
  - Hence it is more likely that an arrival finds a free server.
- Even in an M/M/1, if we hold ρ constant, but increase both λ and μ by the same factor, then delay will drop indefinitely



M/M/∞

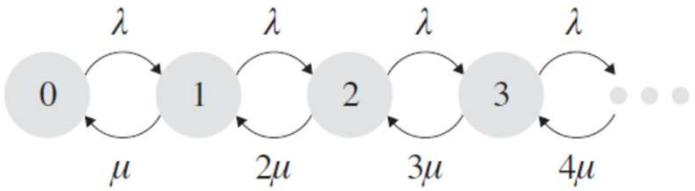


Figure 15.1. CTMC for the M/M/∞ system.

- Imagine that you could call up the phone company for customer service, and never get the message, "We're sorry, but all of our service representatives are busy serving other customers . . . "

$$\pi_1 = \frac{\lambda}{\mu}\pi_0$$

$$\pi_2 = \frac{\lambda}{2\mu}\pi_1 = \frac{\lambda}{2\mu}\cdot\frac{\lambda}{\mu}\cdot\pi_0$$

$$\pi_3 = \frac{\lambda}{3\mu}\pi_2 = \frac{\lambda}{3\mu}\cdot\frac{\lambda}{2\mu}\cdot\frac{\lambda}{\mu}\cdot\pi_0$$

$$\pi_i = \left(\frac{\lambda}{\mu}\right)^i \cdot \frac{1}{i!}\cdot\pi_0.$$

$$\pi_0 = e^{-\frac{\lambda}{\mu}}$$

$$\text{Number of jobs in the M/M/}\infty \sim \text{Poisson}\left(\frac{\lambda}{\mu}\right)$$

$$\mathbf{E}[N] = \frac{\lambda}{\mu}.$$

$$\mathbf{E}[T] = \frac{\mathbf{E}[N]}{\lambda} = \frac{1}{\mu}.$$

CAPACITY PROVISIONING FOR M/M/K

- If the average arrival rate is λ, and the average service rate at each server is μ, what is the minimum number of servers, k, needed just to keep the system stable?
  - Resource Requirement, $R = \frac{\lambda}{\mu}$
- Square-Root Staffing Rule:

  Given an M/M/k with arrival rate λ and server speed μ and $R = \lambda/\mu$, where R is large, let k_α^* denote the least number of servers needed to ensure that $P_Q < \alpha$.

  Then $k_\alpha^* \approx R + c\sqrt{R}$, where c is the solution to the equation, $\frac{c\Phi(c)}{\phi(c)} = \frac{1-\alpha}{\alpha}$

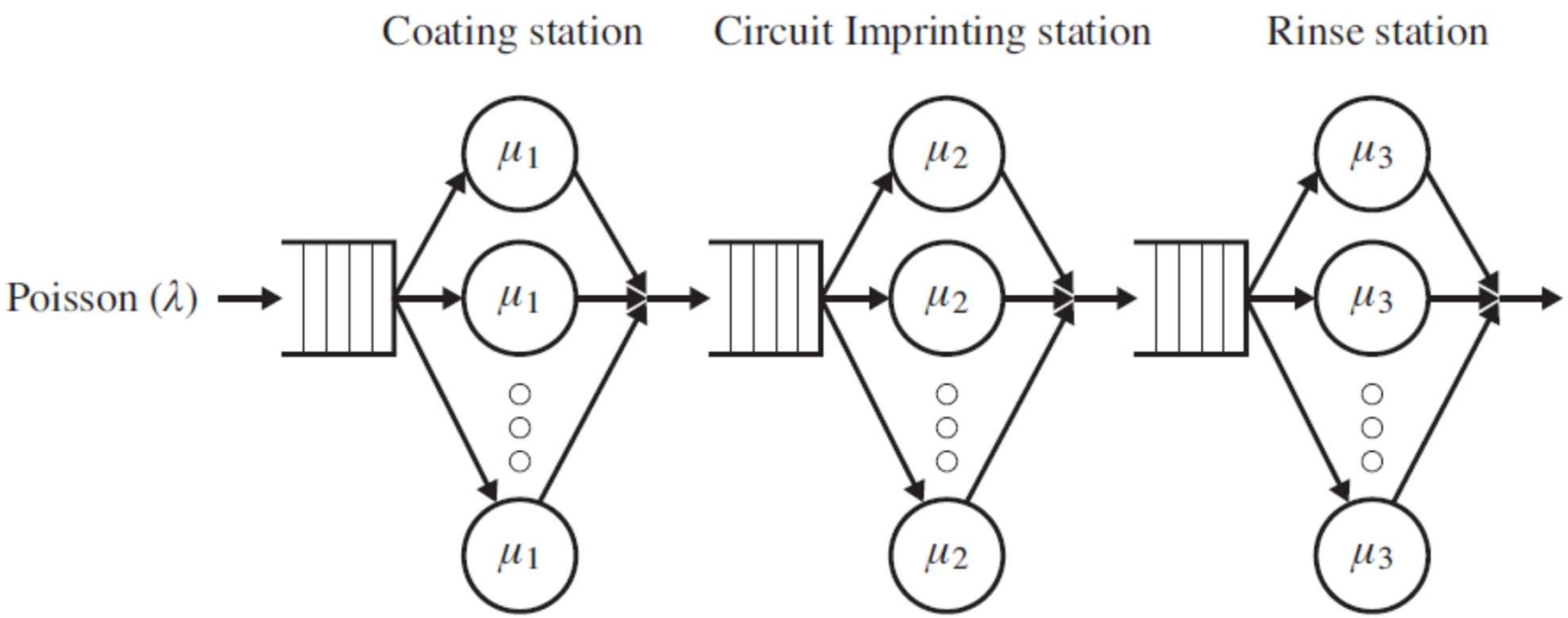
  where $\Phi(\cdot)$ denotes the c.d.f. of the standard Normal and $\varphi(\cdot)$ denotes its p.d.f.

$\alpha = 0.8$	$\alpha = 0.5$	$\alpha = 0.2$	$\alpha = 0.1$
$c = 0.173$	$c = .506$	$c = 1.06$	$c = 1.42$

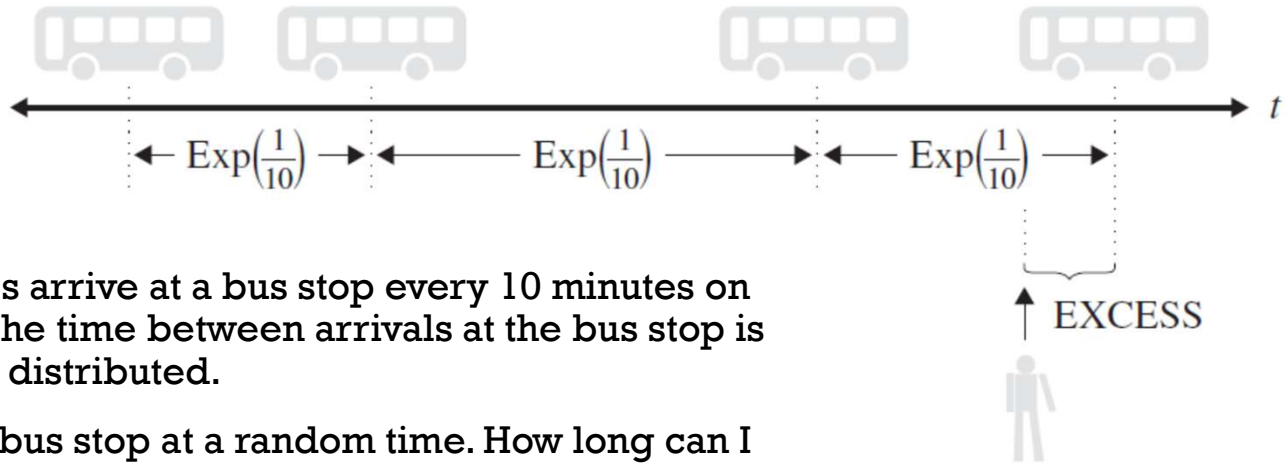
This rule works well even for staffing small server farms!!



NETWORK OF QUEUES



INSPECTION PARADOX



- Suppose buses arrive at a bus stop every 10 minutes on average, and the time between arrivals at the bus stop is Exponentially distributed.
- I arrive at the bus stop at a random time. How long can I expect to wait for a bus?

Definition 23.1 Let A denote the time between bus arrivals. Let's suppose a person arrives at a random time. Then the time that person has to wait until the next bus is denoted by the random variable A_e and is called the ***excess of A***.

INSPECTION PARADOX

- Average waiting time at the bus stop: $\mathrm{E}[\mathrm{S_e}] = \frac{E[S^2]}{2E[S]}$
- Expected Time since the last bus arrived: $\mathrm{E}[\text{age of S}] = \frac{E[S^2]}{2E[S]}$
- Expected Time between buses, as seen by the arrival: $\frac{E[S^2]}{E[S]}$
- If S is highly variable, this quantity can be far greater than $E[S]$

$$C_S^2 = \frac{\mathbf{Var}(S)}{\mathbf{E}[S]^2} = \frac{\mathbf{E}[S^2]}{\mathbf{E}[S]^2} - 1,$$

$$\mathbf{E}[S_e] = \frac{\mathbf{E}[S^2]}{2\mathbf{E}[S]} = \frac{\mathbf{E}[S]}{2} \cdot \frac{\mathbf{E}[S^2]}{\mathbf{E}[S]^2} = \frac{\mathbf{E}[S]}{2} \cdot (C_S^2 + 1).$$

In practical terms, you arrive at the bus stop and find that the time until the next bus is many times higher than the mean time between buses.



ANALYSIS OF M/G/1 QUEUE

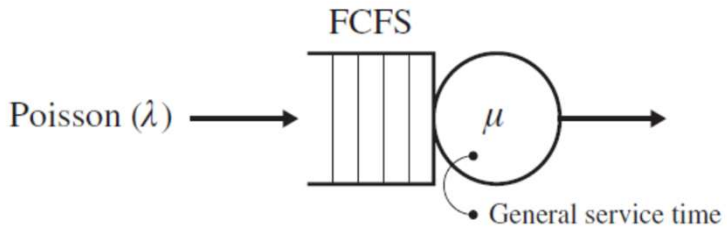


Figure 23.2. An M/G/1 queue.

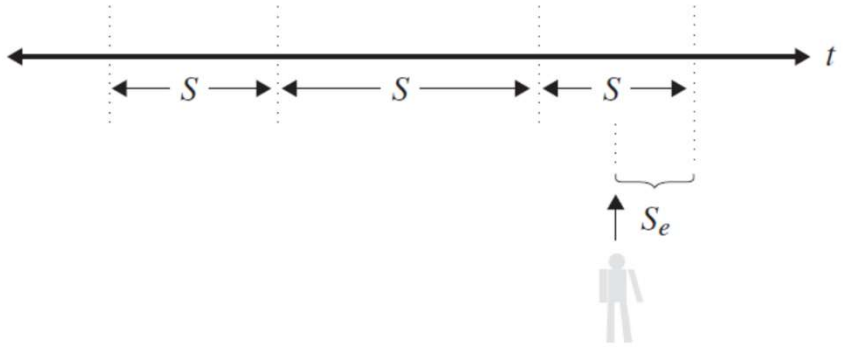
- T_Q: time in queue
- N_Q : number in queue
- N_Q^A: number in queue as seen by the arrival
- S : service time of a job, where $\mathbf{E}[S] = 1/\mu$
- S_i: service time of the ith job in the queue
- S_e : excess of S – the remaining service time of the job in service, *given* that there is some job in service.

M/G/1 CONT..

$$\begin{aligned}\mathbf{E}[T_Q] &= \mathbf{E}[\text{Unfinished work that an arrival witnesses in the system}] \\ &= \mathbf{E}[\text{Unfinished work in queue}] + \mathbf{E}[\text{Unfinished work at the server}] \\ &\quad (\text{Expectations add even if r.v.s are not independent.})\end{aligned}$$

$$= \mathbf{E}\left[\sum_{i=1}^{N_Q^A} S_i\right] + \mathbf{E}[\text{Unfinished work at server}]$$

$$= \mathbf{E}[N_Q^A] \cdot \mathbf{E}[S] + \mathbf{P}\{\text{Arrival sees job in service}\} \cdot \mathbf{E}[S_e]$$
$$= \mathbf{E}[N_Q] \mathbf{E}[S] + (\text{Time-avg probability server busy}) \cdot \mathbf{E}[S_e]$$
$$= \mathbf{E}[N_Q] \cdot \mathbf{E}[S] + \rho \cdot \mathbf{E}[S_e]$$
$$= \mathbf{E}[T_Q] \cdot \lambda \cdot \mathbf{E}[S] + \rho \cdot \mathbf{E}[S_e]$$
$$= \mathbf{E}[T_Q] \cdot \rho + \rho \cdot \mathbf{E}[S_e]$$
$$= \frac{\rho}{1-\rho} \cdot \mathbf{E}[S_e].$$



$$E[T_Q] = \frac{\rho}{1-\rho} E[S_e]$$

M/G/1 EXAMPLES

- **M/M/1 Queue:** The service time, S, is Exponentially distributed with mean $1/\mu$. Because the service time distribution is memoryless, $\mathbf{E}[S_e] = 1/\mu$. Therefore,

$$\mathbf{E}[T_Q] = \frac{\rho}{1-\rho} \cdot \frac{1}{\mu}.$$

- **M/D/1 Queue:** The service time is Deterministic (constant) and equal to $1/\mu$. $\mathbf{E}[S_e] = \frac{1}{2\mu}$, because the remaining service time of a job in service is Uniformly distributed between 0 and $\frac{1}{\mu}$. Therefore,

$$\mathbf{E}[T_Q] = \frac{\rho}{1-\rho} \cdot \frac{1}{2\mu}.$$

Note that the expected time in queue is *half* that of the M/M/1 queue.



ERLANG-K DISTRIBUTION

M/E_k/1 Queue: The service time has an Erlang-k distribution (see Definition 21.2). The E_k distribution consists of k stages in series, each with Exponential service time with mean $1/k\mu$. If there is a job in service at the time of an arrival, then it is equally likely that the job is at each of the k stages. On average, the job in service will be at the middle stage, leaving $\lceil \frac{k+1}{2} \rceil$ stages left to be completed. We therefore should have

$$\mathbf{E}[S_e] = \left\lceil \frac{k+1}{2} \right\rceil \cdot \frac{1}{k\mu}.$$

Mean time in queue is then given by

$$\mathbf{E}[T_Q] = \frac{\rho}{1-\rho} \cdot \left\lceil \frac{k+1}{2} \right\rceil \cdot \frac{1}{k\mu}.$$

Observe that for $k = 1$ this is equal to the M/M/1 expression and for $k \to \infty$ this is equal to the M/D/1 expression.



RENEWAL REWARD THEORY

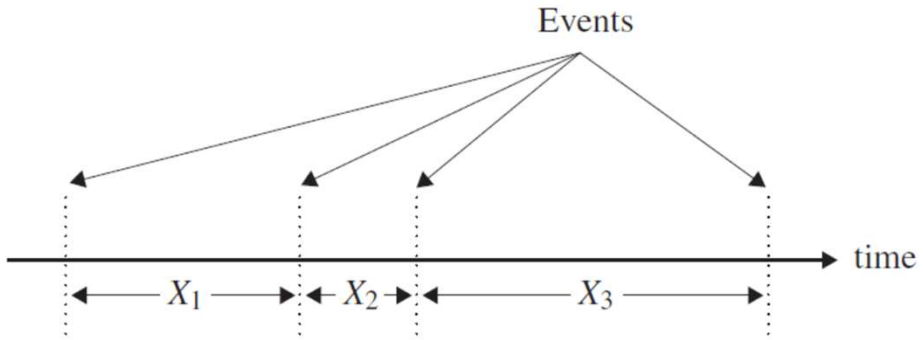


Figure 23.4. A renewal process. The X_i's all have common distribution F.

Theorem 23.3 (restated from Theorem 9.33) *For a renewal process, if* $\mathbf{E}[X] > 0$ *is the mean time between renewals (events), and* $N(t)$ *is the number of renewals (events) by time* t*, then we have, with probability* 1,

$$\frac{N(t)}{t} \to \frac{1}{\mathbf{E}[X]} \quad \text{as } t \to \infty. \tag{23.6}$$

AVERAGE REWARD

Theorem 23.4 (Renewal-Reward) *If* $0 \leq \mathbf{E}[R] < \infty$ *and* $0 < \mathbf{E}[X] < \infty$, *then with probability 1,*

$$\frac{R(t)}{t} \to \frac{\mathbf{E}[R]}{\mathbf{E}[X]} \text{ as } t \to \infty.$$

- The Renewal-Reward theorem says that the average rate at which we earn reward is equal to the expected reward earned during a cycle, divided by the expected cycle length.



USING REWARD TO COMPUTE EXCESS

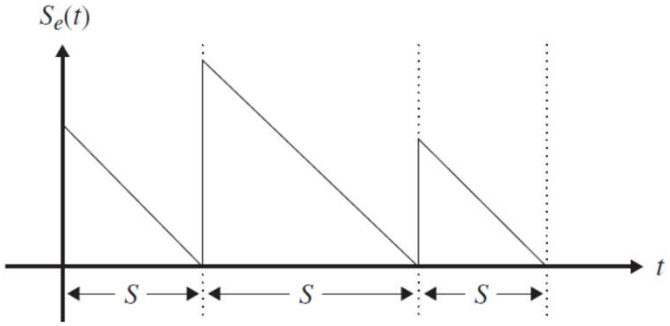


Figure 23.6. The function $S_e(t)$ represents the excess service time at time t.

Reward is the time we wait till the next service starts!

$$\mathbf{E}[S_e] = \text{Time-average Excess} = \lim_{s\to\infty} \frac{\int_0^s S_e(t)dt}{s}.$$

$$\text{Time-average Reward} = \lim_{s\to\infty} \frac{R(s)}{s} = \lim_{s\to\infty} \frac{\int_0^s S_e(t)dt}{s} = \mathbf{E}[S_e]$$

$$\text{Reward earned during a cycle} = \int_0^S (S-t)dt = S^2 - \frac{S^2}{2} = \frac{S^2}{2}$$

$$\mathbf{E}[\text{Reward earned during a cycle}] = \frac{\mathbf{E}[S^2]}{2}$$

$$\mathbf{E}[\text{Length of one cycle}] = \mathbf{E}[S]$$

$$\text{Time-avg Reward} = \frac{\mathbf{E}[\text{Reward during cycle}]}{\mathbf{E}[\text{Cycle length}]} = \frac{\mathbf{E}[S^2]}{2\mathbf{E}[S]}$$

M/G/1 ANALYSIS (P-K FORMULA)

$$\mathbf{E}[T_Q] = \frac{\rho}{1-\rho} \cdot \frac{\mathbf{E}[S^2]}{2\mathbf{E}[S]}$$

$$\mathbf{E}[T_Q] = \frac{\rho}{1-\rho} \cdot \frac{\mathbf{E}[S]}{2} \cdot (C_S^2 + 1)$$

$$\mathbf{E}[T_Q] = \frac{\lambda \mathbf{E}[S^2]}{2(1-\rho)}$$

- What creates delays is the occasional extra-long service time (or extra-large number of arrivals) in some service period.
- Delay is proportional to the *variance* in the *arrival* and *service* processes.
- Once one job is delayed, that affects all jobs in the queue behind it.
- The P-K formula assumes a Poisson arrival process, and the C_S^2 refers only to variability in the service times.



TASK ASSIGNMENT IN SERVER FARMS

- Most server farms (K servers) do immediate dispatching of the jobs to servers via LB
  - They don't have a central queue
- A major question in computer systems design is finding a good task assignment policy to minimize mean response time (or some other performance variant)
- FCFS Policies: Manufacturing Industry, Super Computing, Machine Learning?
  - Jobs are not pre-emptible, have high variability, G, with mean $\frac{1}{\mu}$
  - RANDOM, ROUND-ROBIN, JSQ,M/G/K, LWL, SITA

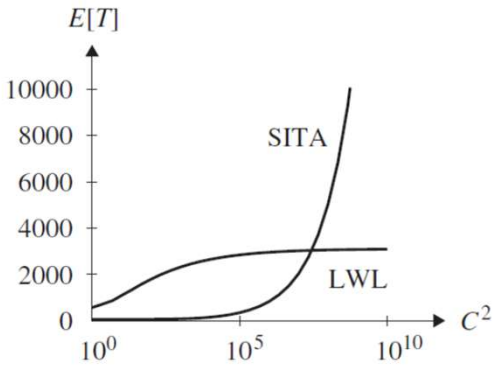


FCFS POLICIES

- Static vs Dynamic Policies
  - Dynamic policies (e.g. JSQ) adapt based on the changes in the state of the system (e.g. number of jobs at each queue); static policies are oblivious to this
- ROUND-ROBIN (RR) slightly outperforms RANDOM scheduling policy
  - RANDOM each queue behaves like an M/G/1, with average arrival rate λ/k,
  - Inter-arrivals in RR is sum of K exponentials (E_K) and has less variability; E_K/G/1
- For high-variability job size distributions, JSQ can lower mean response time by an order of magnitude compared to RR.
  - JSQ balances the instantaneous number of jobs at each queue,
  - RR balances the expected number of jobs
  - RR can cause idling of servers even when some of the other servers are heavily loaded
- M/G/K is even better than JSQ because it can wait on assigning jobs to servers.
  - Under-utilization can never happen under M/G/K



JOB SIZE BASED SCHEDULING



- LWL : Least work left
  - Each job goes to the queue where it will achieve the lowest possible response time. This is a *greedy* policy, because each job is acting in its own best interest
  - As opposed to equalizing the *number* of jobs at each host, the LWL policy aims to equalize the total *work* at each host
  - When job size variability is high, the number of jobs in a queue can be a poor estimate of the total work in that queue. In this sense LWL is far superior to JSQ.
  - LWL and M/G/k are actually equivalent
- SITA: Size interval task assignment
  - Assign to servers based on job sizes
  - Determining optimal cutoffs of the job sizes is extremely difficult!

These policies are better than size agnostic, when job sizes are highly variable!



LOW VARIABILITY CASE

- When job sizes are Deterministic (e.g., all jobs have size 1),
  - ROUNDROBIN policy is optimal, because the arrivals to a server are maximally spaced out;
  - If inter-arrival is deterministic, no job will be delayed under ROUND-ROBIN, if system not overload.
- The JSQ policy will actually end up doing the same thing as ROUND-ROBIN, because the shortest queue will be that which has not received a new job in the longest time.
- By the same logic, LWL will end up doing the same thing as ROUND-ROBIN.
- In contrast, RANDOM will sometimes make the "mistake" of sending two consecutive arrivals to the same queue, incurring some delay.
- SITA will reduce to RANDOM, because all jobs have the same size.
- Even with occasional mistakes by RANDOM, we expect mean response time to be very low.
  - Consider the case of RANDOM with Deterministic job sizes and Poisson arrivals.
  - By Poisson splitting, each queue is M/D/1, which we know has only half the delay of an M/M/1



PS SERVERS

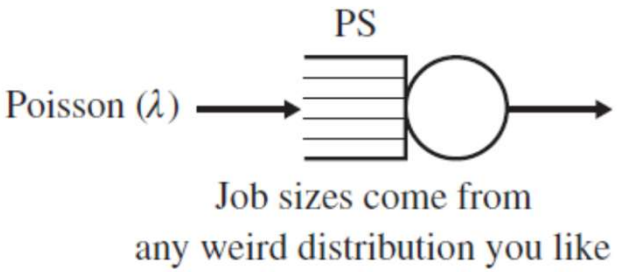


Figure 22.5. M/G/1/PS.



PS SERVER FARMS

- Web server
  - Incoming requests are HTTP requests and must be immediately dispatched to one of the server
  - Requests are fully preemptible in that any request can be stopped and restarted where we left off.
  - Given that this is a network-based application, running on TCP, it is important that the service seem immediate and constant. For this reason, we cannot have requests waiting in a FCFS queue.
  - Instead, each host time-shares among all the requests in its queue, so that each HTTP request receives "constant" service.
- There are many common high-speed routers used for dispatching HTTP requests in a web server farm.
  - Some examples are Cisco's LocalDirector [42], IBM's Network Dispatcher [140], and F5's BIG-IP
  - The job size distribution for websites is known to be highly variable and heavy-tailed



PS SERVER FARMS

- Assume k identical servers and use S to denote the job size, where $\mathbf{E}[S] = 1/\mu$.
- The system load is denoted by $\rho = \lambda/k\mu$ and the resource requirement by $R = \lambda/\mu$.



PERFORMANCE

